

系统架构概要设计说明书——预制菜WMS仓储管理系统

1. 引言

1.1 项目背景

预制菜WMS智能仓储管理系统是为预制菜生产企业量身打造的仓库管理平台。系统覆盖从货物入库、存储管理、质量检查、出库发货到库存盘点的全生命周期，采用 **B/S + 微信小程序** 架构模式，实现前后端分离，支持Web管理端和微信小程序移动端双平台访问。

1.2 设计目标

- 业务闭环**：实现批次创建 → 入库 → 库存管理 → 质检 → 出库 → 报表分析的完整业务链路
- 多端覆盖**：一套后端同时服务Web端(Vue 3)和微信小程序端
- 智能化**：接入DeepSeek大语言模型，实现AI辅助仓储管理
- 数据可视化**：通过ECharts/Canvas图表实时展示仓储运营数据
- 安全可靠**：JWT认证 + BCrypt密码加密 + AES-256传输加密 + 事务保护

1.3 术语定义

术语	说明
WMS	Warehouse Management System, 仓库管理系统
SKU	Stock Keeping Unit, 库存量单位
库位	仓库中的最小存储单元(货架格位)
批次	同一批生产的货物，用批次号标识
FIFO	First In First Out, 先进先出原则
JWT	JSON Web Token, 无状态身份认证令牌

2. 系统总体架构

2.1 架构概览



2.2 技术栈总览

层级	技术	版本	用途
Web前端框架	Vue 3	3.x	用户界面构建
UI组件/图标	Element Plus	latest	图标库
图表	ECharts	5.x	数据可视化看板
HTTP客户端	Axios	latest	前后端通信
构建工具	Vite	latest	开发服务器+打包
小程序框架	微信原生	SDK 3.16.2	移动端
后端框架	Spring Boot	3.x	业务逻辑处理
ORM框架	MyBatis	latest	数据库操作
数据库	MySQL	8.0	业务数据存储
缓存	Redis	6.x	用户会话缓存
认证	JWT (JJWT)	latest	无状态身份认证
密码加密	BCrypt	—	密码存储加密
传输加密	AES-256-CBC	Web Crypto API	密码传输加密
Excel	EasyExcel	latest	数据导入导出
AI模型	DeepSeek	V3	AI智能助手
HTTP客户端(后端)	OkHttp	latest	调用DeepSeek API
API文档	SpringDoc OpenAPI	latest	Swagger自动文档
构建工具(后端)	Maven	—	依赖管理+打包

3. 后端架构设计

3.1 分层架构

后端采用经典的**四层架构**：



3.2 核心模块

模块	关键类	功能
认证鉴权	AuthController, JwtUtil, JwtAuthenticationFilter	登录、JWT生成与验证
用户管理	UserController, UserServiceImpl	用户CRUD、权限、密码加密
仓库管理	WarehouseController, ZoneController, LocationController	仓库→库区→库位三级管理
货物管理	GoodsController	货物信息维护
批次管理	BatchController, BatchServiceImpl	批次创建、状态流转
库存管理	InventoryController	库存查询、详情
入库业务	InboundController, InboundServiceImpl	入库任务：创建→审核→执行→库位分配
出库业务	OutboundController, OutboundServiceImpl	出库任务：创建→审核→执行→库存扣减
质检业务	QualityCheckController, QualityCheckServiceImpl	质检任务：创建→执行→结果处理
盘点业务	InventoryCheckController, InventoryCheckServiceImpl	盘点任务：按范围创建→录入实盘数→差异处理
调整业务	AdjustmentController, AdjustmentServiceImpl	库存移库：选源→选目标→执行移动

模块	关键类	功能
次品处理	<code>DefectiveController</code> , <code>DefectiveServiceImpl</code>	报废处理：锁定→删除库存→级联删除
任务管理	<code>WorkTaskController</code>	工作任务CRUD、我的任务查询
操作日志	<code>OperationLogController</code>	日志记录、筛选查询、Excel导出
AI助手	<code>AiController</code> , <code>AIServiceImpl</code>	DeepSeek对话、真实数据注入、首页统计
数据看板	<code>DashboardController</code> , <code>DashboardServiceImpl</code>	4个图表数据API、SQL聚合查询
温湿度	<code>TemperatureHumidityRecordController</code>	温湿度记录上传与查询

3.3 统一响应格式

所有API返回统一JSON结构 `ResultData extends HashMap<String, Object>`:

```
{
  "code": 200,
  "msg": "操作成功",
  "data": { ... },
  "token": "eyJhbG..."
}
```

状态码定义 (`ResultCode` 枚举):

Code	含义
200	操作成功
500	操作失败
1001	用户名或密码错误
1002	令牌已过期或失效
1003	业务处理异常

3.4 JWT认证流程

```
用户登录 → POST /auth/login
↓
AuthServiceImpl: BCrypt验证密码 → JwtUtil.generateToken(userId, username)
↓
返回 { user, token } → 前端存localStorage
↓
后续请求: Authorization: Bearer <token>
↓
JwtAuthenticationFilter.doFilterInternal()
→ 白名单路径放行 (/auth/login, /register, /swagger-ui/**)
→ 其他路径: 提取token → 验证有效性 → 注入userId到request
→ token无效: 返回 {"code":1002, "msg":"令牌已过期"}
```

JWT配置: 密钥签名(HMAC-SHA384), 2小时过期, 载荷包含userId和username.

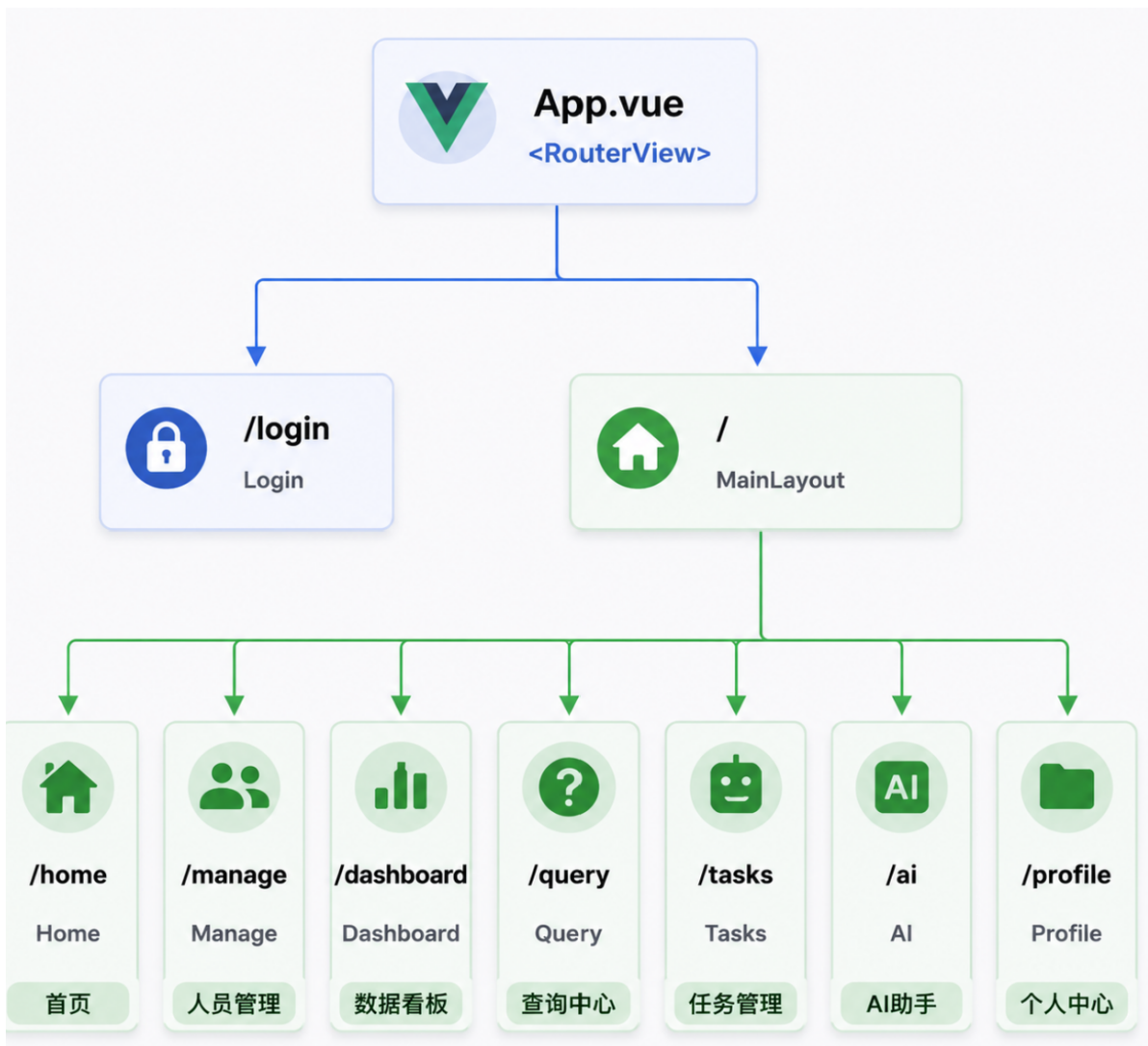
3.5 全局异常处理

`GlobalExceptionHandler` (@RestControllerAdvice):

- `BusinessException` → 业务异常(1003)
- `MethodArgumentNotValidException` → 参数校验失败
- `Exception` → 兜底处理(500)

4. 前端架构设计 (Web端)

4.1 路由架构



- **路由模式**: Hash模式 (`createWebHashHistory`)
- **路由守卫**: `beforeEach` 检查token(无token→/login); 主管权限检查(`requirePosition: '主管'`)
- **懒加载**: Dashboard页面使用动态import(`() => import(...)`)优化首屏加载

4.2 组件架构 (MainLayout)

```
MainLayout.vue
├── .sidebar (可折叠: 224px ↔ 60px)
│   ├── .logo (WMS品牌)
│   ├── .nav (8个菜单项)
│   │   ├── 主页 (HomeFilled图标)
│   │   ├── 我的管理 (主管可见, 3个子项)
│   │   ├── 数据看板 (懒加载)
│   │   ├── 明细查询 (6个Tab子项)
│   │   ├── 工作任务 (2个Tab子项)
│   │   ├── AI助手
│   │   └── 个人中心
│   └── .sidebar-bottom (退出登录)
└── .main
    ├── .topbar (面包屑 + 主题切换 + 用户头像)
    ├── .content-area
    │   └── <RouterView />
    └── .layout-footer
```


4.3 状态管理

不使用Vuex/Pinia。全局状态通过以下方式管理：

- **localStorage**: token, userInfo, ai_session_id, sidebarCollapsed, app-theme
- **路由参数**: 跨页面数据传递(query params)
- **组件data()**: 每个页面管理自己的数据

4.4 HTTP通信

Axios封装(`utils/request.js`):

- `baseUrl: '/api'` → Vite代理转发到 `localhost:8088`
- 请求拦截器: 自动附加 `Authorization: Bearer <token>`
- 响应拦截器: 自动提取 `response.data`; Excel导出特殊处理(blob下载)
- 超时: 10秒

5. 微信小程序架构

5.1 页面结构

```
App (app.js)
├─ TabBar页面 (4个)
│   ├─ pages/home/home          (首页)
│   ├─ pages/inventory/         (库存)
│   ├─ pages/tasks/tasks        (任务)
│   └─ pages/profile/profile    (我的)
├─ 业务页面 (9个)
│   ├─ pages/inbound/           (入库管理)
│   ├─ pages/inbound-create/    (创建入库)
│   ├─ pages/outbound/          (出库管理)
│   ├─ pages/outbound-create/   (创建出库)
│   ├─ pages/quality/           (质检管理)
│   ├─ pages/quality-create/    (创建质检)
│   ├─ pages/goods/             (货物管理)
│   ├─ pages/batch/             (批次管理)
│   └─ pages/batch-create/      (创建批次)
├─ 功能页面 (3个)
│   ├─ pages/login/             (登录)
│   ├─ pages/env-monitor/       (环境监控)
│   └─ pages/dashboard/         (数据看板)
└─ 基础页面 (1个)
    └─ pages/warehouse/         (仓库)
```

5.2 小程序通信

- **HTTP请求**: `utils/api.js` (旧版Promise包装) + `wx.request`
- **自动附加Token**: 从 `app.globalData.token` 读取
- **缓存穿透防护**: GET请求自动添加 `_t` 时间戳参数
- **本地存储**: `wx.getStorageSync` 存储token和userInfo

5.3 小程序权限

客户端权限检查（基于 `userInfo.position`）：

- 仓库管理员 / 管理部门 → 管理员 (可创建出入库/质检订单)
- 仓库员工 → 员工 (可执行出入库操作)
- 质检员 → 质检员 (可执行质检操作)

6. 数据库架构

6.1 数据库概览

- 数据库名： `pmf`
- 引擎：InnoDB
- 字符集：utf8mb4
- 表数量：22张

6.2 核心实体关系



6.3 表分类

类别	表	数量
基础数据	user, goods, warehouse, zone, location	5
业务数据	batch, inventory	2
入库单据	inbound_order_head, inbound_order_detail	2
出库单据	outbound_order_head, outbound_order_detail	2
调整单据	adjustment_order_head, adjustment_order_detail	2
盘点单据	inventory_check_head, inventory_check_detail	2

质检单据	quality_check_head, quality_check_detail	数量
次品处理	defective_handling_head, defective_handling_detail	2
辅助表	work_task, operation_log, temperature_humidity_record	3

7. 核心业务流程设计

7.1 入库流程

创建批次(B)→ 质检通过(C) → 创建入库单(I) → 员工执行入库

- 选择库位(按储存条件过滤) → 确认执行
- 系统操作(10张表):
 1. inbound_order_head - 状态→已完成
 2. inbound_order_detail - 记录明细
 3. work_task - 标记完成
 4. operation_log - 记录操作
 5. inventory - 创建库存记录(状态→正常)
 6. location - 标记库位占用
 7. zone - 减可用库位
 8. warehouse - 减可用库位
 9. goods - 加库存数量
 10. batch - 状态→正常

7.2 出库流程

创建出库单(O) → 员工执行出库

- 确认出库
- 系统操作:
 1. inventory - 扣减库存(quantity - n)
 2. location - 库存清空则释放库位
 3. zone/warehouse - 恢复可用库位
 4. goods - 减总数量
 5. batch - 减remaining_quantity(归零则级联删除: batch → QC详情/盘点详情/出库详情/库存)

7.3 质检流程

创建质检单(C) → 质检员执行 → 选择结果

- 合格: batch.status = '正常'
- 不合格: batch.status = '报废', inventory.status = '待报废'

7.4 核心设计模式

单据编号规则: 前缀 + yyyyMMdd + 3位流水号

- I → 入库单 (Inbound)
- O → 出库单 (Outbound)
- C → 质检单 (Check)
- P → 盘点单 (check)
- A → 调整单 (Adjustment)

- D → 次品处理单 (Defective)
- B → 批次 (Batch)

库位锁定机制： 出库/调整操作期间锁定源库位，防止并发冲突。

`location.lock_status = '锁定', location.lock_purpose = '出库'/'调整'`

库位分配算法： 每库位存放10个单位，最后剩余部分放最后一个库位。

`所需库位数 = ceil(初始数量 / 10)`

8. AI助手架构

8.1 技术方案

用户输入问题

↓

POST /ai/chat { question, history }

↓

AIServiceImpl.chat()

├ 构建系统提示词（定义AI人设，限制回答范围）

├ buildDataContext(question) ← 关键词匹配，查询真实数据库

| ├ "库存" → Inventory表

| ├ "货物" → Goods表

| ├ "仓库" → Warehouse表

| ├ "批次" → Batch表

| └ "任务" → WorkTask表

├ 组装消息：system_prompt + data_context + history + question

└ OkHttp → DeepSeek API (<https://api.deepseek.com/v1/chat/completions>)

↓

返回AI回复（基于真实数据）

8.2 系统提示词

- 定义角色："预制菜仓储管理(WMS)智能助手"
- 限制范围：仅回答仓储相关问题
- 严禁编造数据：要求引用注入的真实数据
- 安全边界：拒绝娱乐/金融/政治等无关问题
- 免责声明：每个回复末尾附加

9. 安全架构

9.1 认证安全

层级	方案	说明
密码存储	BCrypt单向加密	不可逆，登录时比对密文
密码传输	AES-256-CBC	前端加密后传输，Base64编码
会话管理	JWT (HMAC-SHA384)	2小时过期，无状态
缓存会话	Redis	30分钟TTL，减少DB查询

9.2 权限安全

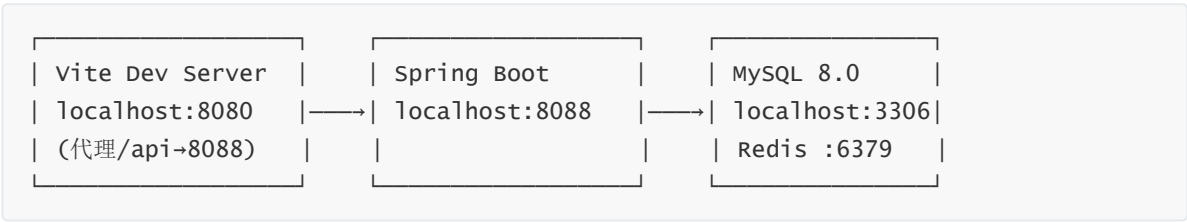
层级	方案	说明
Web路由	<code>router.beforeEach</code>	检查token + position
Web菜单	<code>visibleMenuItems</code>	主管才显示管理菜单
Web按钮	<code>isAdmin/isStaff/isInspector</code>	条件渲染
小程序	<code>wx:if</code> 条件渲染	基于userInfo.position
后端	(当前仅前端校验)	待加强

9.3 数据安全

- **事务保护**: `@Transactional` 保证多表操作的原子性
- **CORS**: `CorsConfig` 配置跨域白名单(当前开发阶段全允许)
- **SQL注入防护**: MyBatis参数化查询 `#{param}`

10. 部署架构

10.1 开发环境



10.2 关键技术配置

后端 (application.yaml):

```
server.port: 8088
spring.datasource.url: jdbc:mysql://127.0.0.1:3306/pmf
spring.data.redis.host: localhost
deepseek.api.key: (配置于application.yaml)
deepseek.api.url: https://api.deepseek.com/v1/chat/completions
```

前端 (vite.config.js):

```
server: {
  port: 8080,
  proxy: {
    '/api': { target: 'http://localhost:8088', changeorigin: true }
  }
}
```

11. 技术决策说明

决策	选择	理由
前端框架	Vue 3 Options API	学习曲线平缓，适合课程项目
UI方案	自定义CSS + Element Plus Icons	轻量灵活，避免组件库锁定
状态管理	不使用Pinia/Vuex	项目规模适中，localStorage+props满足需求
后端框架	Spring Boot 3	生态成熟，自动配置减少样板代码
ORM	MyBatis (XML)	复杂SQL灵活控制，动态SQL支持好
缓存	Redis	用户会话缓存，降低DB压力
AI接入	DeepSeek (OpenAI兼容)	性价比高，中文能力强
小程序	原生框架	避免额外学习成本，直接使用微信API
构建	Maven + Vite	主流选择，社区支持好